Reproducing and Critiquing "F1 > 0.999" on CICIDS2017

A Methodological Audit of Rodríguez et al. (2022)

Eyal Steinmetz
Data Science Methods in Cyber Security — Dr. Uri Itai
University of Haifa

July 2026

Abstract

Rodríguez et al. (2022, *Sensors* MDPI) report that tree-based classifiers reach **F1 values above 0.999** on the CICIDS2017 intrusion-detection dataset. We reproduce their Random Forest pipeline in Python/scikit-learn, *confirm* the headline (macro-F1 = 0.9987 under a random split on the raw, concatenated data), and then show that this number is an artifact of methodology rather than evidence of a deployable detector. Removing duplicate flows barely changes the binary headline (+0.03 pp), but a leakage decomposition reveals *why*: the random split is trivially easy because every attack *type* appears in both training and test. Under an honest temporal split (train Mon–Thu, test Friday), macro-F1 collapses to 0.4883 (MCC 0.2541) — a total drop of **51.0 percentage points** — because 100% of Friday's attack flows belong to classes never seen in training. Finally, rare attacks (Heartbleed, Infiltration, SQL Injection) have only 2–7 evaluable test samples, so a single aggregate metric is meaningless for them. We conclude that the author's F1 > 0.999 claim is *not supported* as evidence of generalisation, in agreement with Engelen et al. (2021) and Lanvin et al. (2022).

Contents

1 Summary of the Source

Problem and its importance. Network Intrusion Detection Systems (IDS) classify network traffic flows as benign or malicious. The problem is central to defensive cybersecurity: a Security Operations Center (SOC) relies on an IDS to surface attacks early, and the cost of a missed attack (false negative) is typically far higher than that of a false alarm (false positive). Flow-based detection — classifying statistical summaries of connections rather than raw packets — is attractive because it scales to high-throughput networks. The central research question is whether supervised machine learning can detect intrusions reliably from such flow features.

The selected source. We critique a *peer-reviewed journal article*: Rodríguez, M., Alesanco, Á., Mehavilla, L., & García, J. (2022), "Evaluation of Machine Learning Techniques for Traffic Flow-Based Intrusion Detection," *Sensors* **22**(23), 9326, MDPI (DOI `10.3390/s22239326`) [1]. Choosing a published, indexed paper (rather than a blog or notebook) makes the critique more serious: we are testing the validity of a claim that passed peer review.

Proposed solution and methodology. The authors evaluate a battery of classifiers in **Weka** (Naive Bayes, Logistic Regression, MLP, SMO, KNN, AdaBoost, OneR, J48, PART, and Random Forest) on a single "joint file" obtained by concatenating all five days of CICIDS2017 (2,830,743 flows). Models are trained and tested with a *random* train/test split over that joint file, and performance is reported as a **macro-averaged F1 score**. Their headline finding is:

> "tree-based techniques (PART, J48, and random forest) have turned out to be the most efficient with **F1 values above 0.999** (average obtained in the complete dataset)."

The dataset. CICIDS2017 [4] contains five working days (Monday–Friday) of captured traffic, with flow features extracted by the CICFlowMeter tool. It comprises 2,830,743 flows, one `BENIGN` class and 14 attack types, and is heavily imbalanced (BENIGN $\approx 80\%$). Crucially, the attacks are *segregated by day*: e.g. DoS attacks occur on Wednesday, web attacks and Infiltration on Thursday, and DDoS/PortScan/Botnet on Friday. This temporal structure is the lever for our critique.

2 Critical Evaluation of the Author's Claims

This section is the core of the report. Following the principle CLAIM → OUR TEST → OUR RESULT → VERDICT, we never criticise without our own measured experiment. All numbers below come from our notebook and are exported verbatim to `notebooks/metrics_export.json`.

2.1 Claim 1: reproducibility of F1 > 0.999

Claim. A Random Forest achieves macro-F1 > 0.999 on the complete dataset.
Our test (Strategy A0). We replicate the authors' protocol exactly: raw data (duplicates kept), a stratified random 80/20 split over all days concatenated, a `StandardScaler` fit on the training split only, and a near-default `RandomForestClassifier` (100 trees, `random_state=42`).
Our result. Macro-F1 = **0.9987**, accuracy = 0.9992.
Verdict: supported. The headline is reproducible; the authors' *method* is replicable. The disagreement is not about the number but about what it *means*.

2.2 Claim 2: the result reflects detector quality (it does not)

The authors implicitly treat the 0.999 as evidence that the detector works. We dispute this through three documented flaws, mapped to Engelen et al. (2021) [2].

Flaw 1 — duplicate contamination, and why it is *not* the lever.

CICIDS2017 contains a large number of exact-duplicate flows (307,078 rows, 10.86% of the raw data; the deduplication mask used by the models, taken over the engineered feature set, flags 307,991 rows / 10.89%). A random split scatters copies of the same flow across train and test, which *could* let a model memorise test points [2]. We test this directly (Strategy A1): deduplicate, then re-run the identical random pipeline.

The binary macro-F1 barely moves: 0.9987 → 0.9984, an inflation of only +0.0003 (+0.03 pp). To understand why, we decompose the A0 test set (Figure 1): 13.4% of A0 test rows are exact duplicates of a training row, yet accuracy on those *leaked* rows (0.9994) is essentially equal to accuracy on *unique* test rows (0.9991). **Duplicates are therefore a data-quality footnote, not the inflation mechanism.** This is an honest null result that *strengthens* the critique: the real lever is identified in Flaw 2.

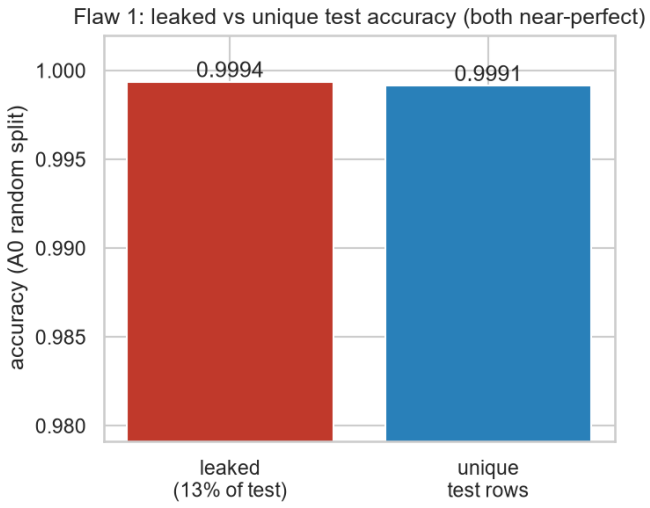

Figure 1: Leakage decomposition of the random split. Leaked (duplicated) and unique test rows are classified with near-identical, near-perfect accuracy: the inflated headline does not come from duplicate memorisation.

Flaw 2 — evaluation methodology: random vs. temporal split.

A random split over a joint file mixes traffic from all days into both train and test. Because CICIDS2017's attacks are day-segregated, this guarantees that every attack *type* is present in both partitions — an *in-distribution* test. A deployed IDS, by contrast, is trained on past traffic and must detect *future* (possibly novel) attacks. We therefore evaluate honestly (Strategy B0): deduplicate, then split temporally (train Monday–Thursday, test Friday), scale on train only, and balance the training set with class-capped multiclass SMOTE before binarising.

The result is decisive (Figure 2): macro-F1 falls from 0.9984 (random, deduped) to **0.4883** (temporal), with accuracy 0.6764, MCC 0.2541, and ROC-AUC 0.7735. The **total gap from the source headline is** $0.9987 - 0.4883 = 0.5104$ **(51.0 percentage points)**. The cause is unambiguous: 100% of Friday's attack flows (DDoS, PortScan, Botnet) belong to classes that never appear in Monday–Thursday training. Per-attack-type recall on Friday confirms the

collapse — DDoS 0.168 (partial transfer from the seen DoS family), PortScan 0.001, Botnet 0.000. Strategy B is thus effectively a near-zero-day generalisation test, and the random split was hiding this entirely.

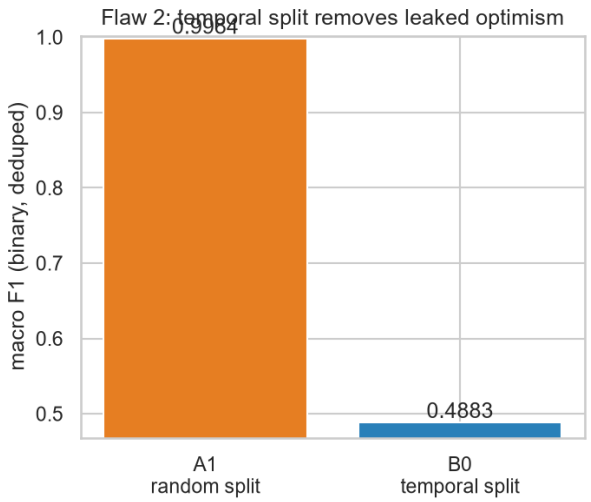

Figure 2: Binary macro-F1 under a random split vs. an honest temporal split. The 51-pp gap is the optimism removed by respecting the temporal order of attacks.

Flaw 3 — a macro average hides rare-class failure.

Reporting a single macro-F1 on a dataset that mixes 230k-sample classes with 11-sample classes is statistically misleading. Using a stratified deduped multiclass model, the rare classes have only 2–7 test samples each: Heartbleed recall 1.000 (support 2), Infiltration 0.714 (support 7), SQL Injection 0.250 (support 4). These numbers are *un-evaluable*: any apparent "hit" is a memorised near-duplicate, and a single aggregate number conceals that rare-but-dangerous attacks cannot be assessed at all.

2.3 Are the conclusions justified? Verdict table

Source claim	Our test	Our result	Supported?
F1 $\approx$ 0.999 (their method)	raw + random split	A0 F1 = 0.999	Yes (reproduced)
0.999 holds after dedup	deduped + random split	A1 F1 = 0.998	Yes; leakage hidden
0.999 holds in deployment	deduped + temporal split	B0 F1 = 0.488, MCC 0.254	**No**
Effective on all attack types	per-class recall	PortScan/Botnet recall $\approx$ 0; rare classes un-evaluable	**No**

Table 1: Claim-by-claim verdict. The headline reproduces, but every claim about *generalisation* fails under sound evaluation.

Limitations of the source. The paper (i) does not mention deduplication; (ii) uses a random split on temporally-structured data; (iii) reports only an aggregate metric, with no per-class breakdown for rare attacks; and (iv) reports no confidence intervals or temporal robustness analysis. None of these are fatal individually, but together they make the 0.999 headline an in-distribution memorisation score rather than evidence of a usable detector.

3 Exploratory Data Analysis

Before modelling, we explored the data to motivate every downstream decision.

Class imbalance and its real-world meaning. BENIGN traffic is 80.3% of flows; the binary attack rate is 19.68% (Figure 3). In a real network the benign fraction is usually *higher* still, so the imbalance is realistic in direction but the attack prevalence here is inflated by the controlled capture. Imbalance means accuracy is an unreliable headline and motivates recall-aware metrics (MCC, F_β) and resampling (SMOTE) on the training split only. The source did not discuss imbalance handling.

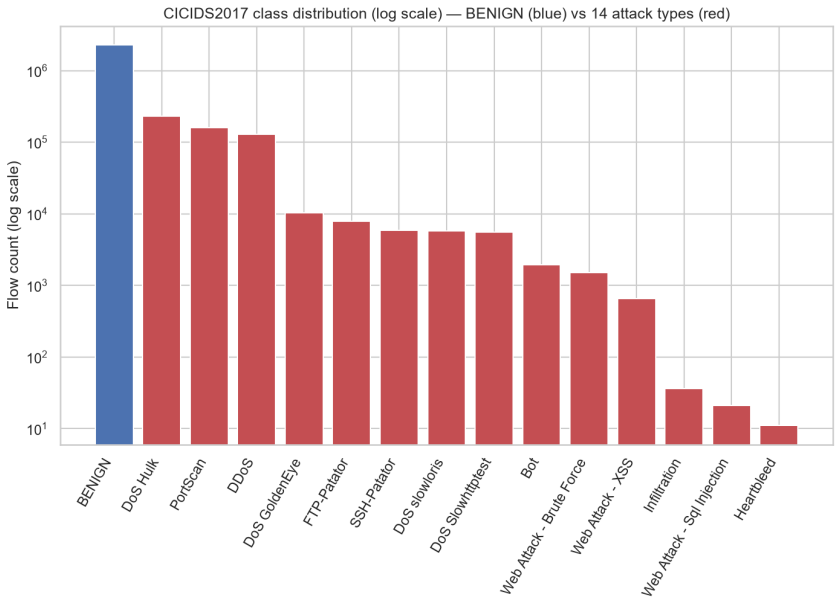

Figure 3: Class distribution (log scale). BENIGN dominates; rare attacks such as Heartbleed, SQL Injection, and Infiltration have a handful of samples each.

Temporal structure. Attacks are segregated by day (Figure 4): this is the empirical basis for the temporal-split critique. A data-driven cross-tab corrected the assumed mapping (Botnet, DDoS, and PortScan are Friday phenomena). Because no clean temporal split can share attack *types* between train and test, Strategy B is necessarily a cross-attack-type test.

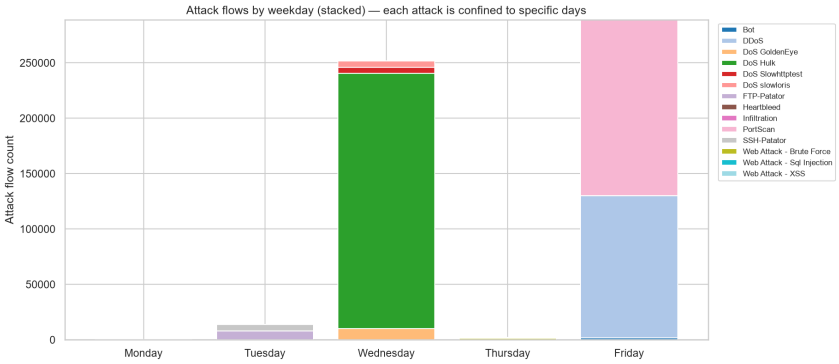

Figure 4: Attack composition by weekday. Each attack family is concentrated on specific days, so a random split leaks attack-type membership across train and test.

Quality issues, distributions, and correlation. We found 4,376 `Inf` cells (in `Flow Bytes/s` and `Flow Packets/s`, a known CICFlowMeter artifact [2]), leading/trailing whitespace in column names, a duplicated `Fwd Header Length.1` column, and 8 zero-variance columns. Most features are heavily right-skewed (52 of 78 features have skewness > 3), motivating a `log1p` transform. For redundancy we used **Spearman** rank correlation rather than Pearson, because the features are non-normal and outlier-heavy and we care about monotonic (not strictly linear) association; Spearman is $O(n \log n)$ and robust to outliers, whereas Kendall is prohibitively expensive on millions of rows. Many feature pairs exceed $|r| > 0.95$ (e.g. Subflow $\approx$ Total byte counts, segment-size $\approx$ packet-length families), which we prune in feature engineering.

4 Feature Engineering Analysis

Was feature engineering performed, and which features were used? The raw data has 78 numeric flow features after dropping the day/label tags. We engineered a 40-feature matrix through the steps below; the binary label is `(Label` $\neq$ `BENIGN)` and the multiclass label is a 15-class encoding.

Transformations and their rationale.

- **Cleaning:** `Inf`→`NaN` (CICFlowMeter artifact), column whitespace stripping, label-encoding fixes for a non-UTF-8 dash in web-attack labels.
- **Structural removal:** dropped the duplicated `Fwd Header Length.1` column and 8 zero-variance columns (no information, pure noise/dimensionality).
- **Log transform:** `log1p` applied to 23 right-skewed non-negative features (7 skewed-but-negative features excluded). Skew > 3 degrades distance/variance-based steps; `log1p` is stateless, so applying it per split cannot leak.
- **Scaling:** `StandardScaler` *fit on the training split only*, then applied to test — the canonical no-leakage protocol.
- **Encoding:** the only categorical is the target, handled with `LabelEncoder`; features are already numeric, so no one-hot encoding is needed.
- **Resampling:** class-capped multiclass SMOTE on the training split only (interpolating within attack families, excluding ultra-rare classes), then binarised — never on combined data, to avoid optimistic leakage.

Redundancy: spotting and tackling it. We spotted redundancy via the Spearman matrix and pruned all features in a correlated pair (over the full candidate set, $|r| > 0.95$): 29 features removed, taking the matrix from 78 to **40** features. This is heavier pruning than a naive estimate because CICIDS2017 encodes the same quantity many ways (Subflow vs. Total, segment-size vs. packet length, Active-time statistics). Redundant collinear features add variance and obscure importance without adding signal.

Was the process meaningful, and what could improve it? Mathematically, removing collinear and zero-variance features reduces variance and improves interpretability; in cybersecurity terms, the surviving features (flow durations, inter-arrival times, packet-length statistics, flag counts) are the behavioural fingerprints that separate scripted attacks from human traffic. Potential additions: entropy of destination ports, windowed connection-rate features per source IP, and payload-derived signals — none of which the flow-only feature set captures, and which would likely help with novel attacks.

5 Reproducibility Analysis

- **Can the code be executed?** Yes. Our notebook runs end-to-end via `nbconvert -execute` on the full 2.83M-row dataset with zero errors and zero warnings, producing all 16 figures and the metrics export.
- **Are files and dependencies available?** Yes. Dependencies are pinned with `uv` (`pyproject.toml` + `uv.lock`); the dataset is the public Kaggle mirror of CICIDS2017, documented in the README and `references/`.
- **Hidden preprocessing?** For *the source*: yes — deduplication is not mentioned and the split strategy is under-specified, which is precisely what allows the inflated headline. For *our work*: no — every step is an explicit, commented cell.
- **Overall.** The source is only *partially* reproducible: it uses Weka with no public code, so the exact pipeline must be inferred. Our Python re-implementation is fully reproducible (fixed seeds, no leakage, conventional commits, `ruff`-clean), which is itself a contribution: it makes the claim falsifiable.
- **Engineering.** The reusable core is factored into a tested package `src/ids_critique/` (`data` / `features` / `evaluate` / `critique`) with **16 passing `pytest` unit tests**; the environment is pinned via `uv.lock` (Python 3.12, scikit-learn 1.9, XGBoost 3.3, imbalanced-learn 0.14). Full version and command details are in Appendix A.

6 Experimental Results

6.1 Experiments and modifications

We ran five experiments: **A0** (raw + random, the source method), **A1** (deduped + random, isolating Flaw 1), **B0** (deduped + temporal + SMOTE, the honest protocol), **Bm** (deduped stratified multiclass, for per-class recall), and two honest alternatives — **XGBoost** and an unsupervised **Isolation Forest**. Our key modification relative to the source is the honest temporal split plus the full, recall-aware metric suite.

6.2 Model training and comparison

All supervised models share the temporal protocol so the comparison is fair (Figure 5). Under that honest split, no supervised model is usable: XGBoost reaches macro-F1 0.4972 (MCC 0.2670, ROC-AUC 0.7425), marginally above the Random Forest (0.4883). The unsupervised Isolation Forest — trained on benign flows only, so it does not depend on having seen an attack class — achieves the best honest *attack recall* (0.328) and binary F1 (0.547), precisely because it treats Friday's novel attacks as anomalies rather than memorised classes.

Model (protocol)	macro-F1	prec.	recall	$F1_+$	F_2	MCC	AUC
RF — A0 (raw + random, *source*)	**0.9987**	0.997	0.999	0.998	0.999	0.997	1.000
RF — A1 (deduped + random)	0.9984	0.997	0.998	0.997	0.998	0.997	1.000
RF — B0 (deduped + temporal)	**0.4883**	0.997	0.098	0.178	0.119	0.254	0.774
XGBoost — B1 (deduped + temporal)	0.4972	0.997	0.108	0.194	0.131	0.267	0.742
Isolation Forest — B2 (unsupervised)	0.5473	0.443	0.328	0.377	0.346	0.106	0.740

Table 2: Full metric suite. *macro-F1* averages both classes (the source's headline); the remaining columns are positive-class (attack) metrics. The honest-protocol rows (B∗) expose the recall collapse that accuracy and the macro headline hide — note B0/B1's high precision (0.997) but ≈0.10 recall: they almost never false-alarm, yet miss ≈90% of attacks.

Friday attack type	flows	recall (B0)	seen in training?
DDoS	128,014	0.168	no (partial transfer from DoS family)
PortScan	90,694	0.001	no
Botnet	1,948	0.000	no

Table 3: Per-attack-type recall on the Friday temporal test (B0). 100% of the day's attack flows belong to classes unseen in Mon–Thu training, which is why recall collapses.

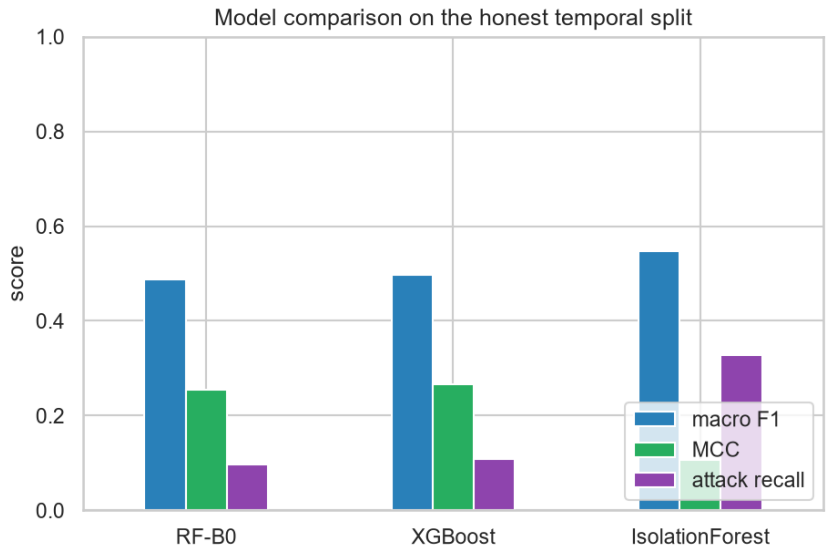

Figure 5: Model comparison on the honest temporal split (F1 / MCC / attack recall). All models are far from the inflated 0.999; the unsupervised model wins on attack recall.

6.3 Evaluation metrics: definitions and why they matter for IDS

We deliberately go beyond the source's single macro-F1. For each metric we give its meaning and its cybersecurity interpretation; the false-positive (FP) vs. false-negative (FN) trade-off is what makes the choice non-trivial.

- **Accuracy** $= \frac{TP+TN}{N}$. *Misleading here*: an "always BENIGN" classifier scores ≈ 0.83 on the Friday test while detecting nothing. We report it only to demonstrate this trap.
- **Precision** $= \frac{TP}{TP+FP}$. The fraction of alerts that are real — governs SOC *alert fatigue*. Low precision floods analysts with false alarms.

- **Recall** $= \frac{TP}{TP+FN}$. The fraction of attacks caught — the most critical IDS metric, since each FN is an undetected intrusion.
- **F1** $= \frac{2PR}{P+R}$ and $\mathbf{F}_\beta$ (β=2), which weights recall higher because missing an attack costs more than a false alarm.
- **MCC** (Matthews correlation, range $[-1, 1]$): robust to imbalance and our preferred single number; B0's MCC = 0.254 exposes that the model is only weakly better than chance, which accuracy (0.676) hides.
- **ROC-AUC / PR-AUC**: threshold-free ranking quality. Under heavy imbalance the precision–recall curve (Figure 6) is more informative than ROC because true negatives dominate.
- **Confusion matrix** (per class, Figure 7): the only view that reveals *which* classes fail.

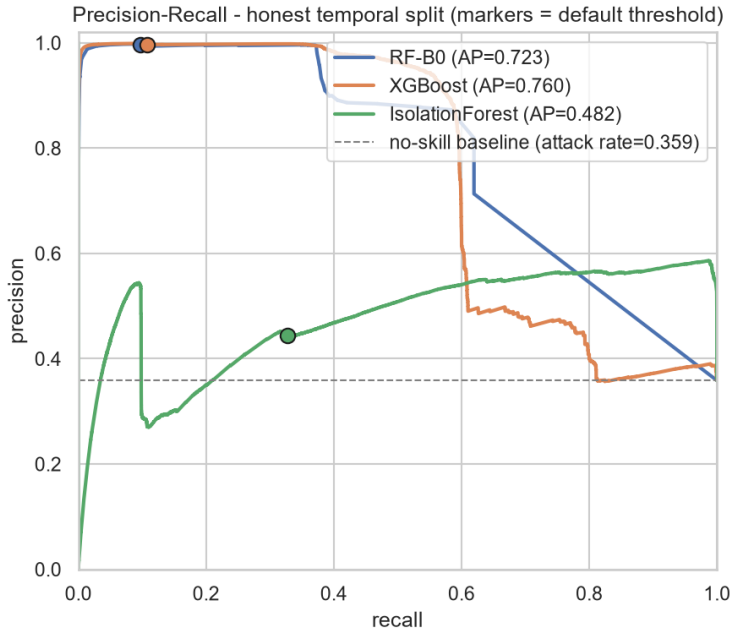

Figure 6: Precision–recall curves on the honest temporal split; markers show each model's default-threshold operating point against the no-skill baseline.

7 Error Analysis

Failures and their patterns. On the Friday temporal test, the Random Forest's false negatives are dominated by the novel attack families: PortScan (99.9% missed) and Botnet (100% missed), with DDoS partially caught (16.8% recall) thanks to transfer from the seen DoS family. The errors are *structural*, not noise: 100% of Friday's attacks are classes unseen in training, so the model has no learned signature for them. The row-normalised multiclass confusion matrix (Figure 7) shows the rare-class rows going dark — the visual signature of Flaw 3.

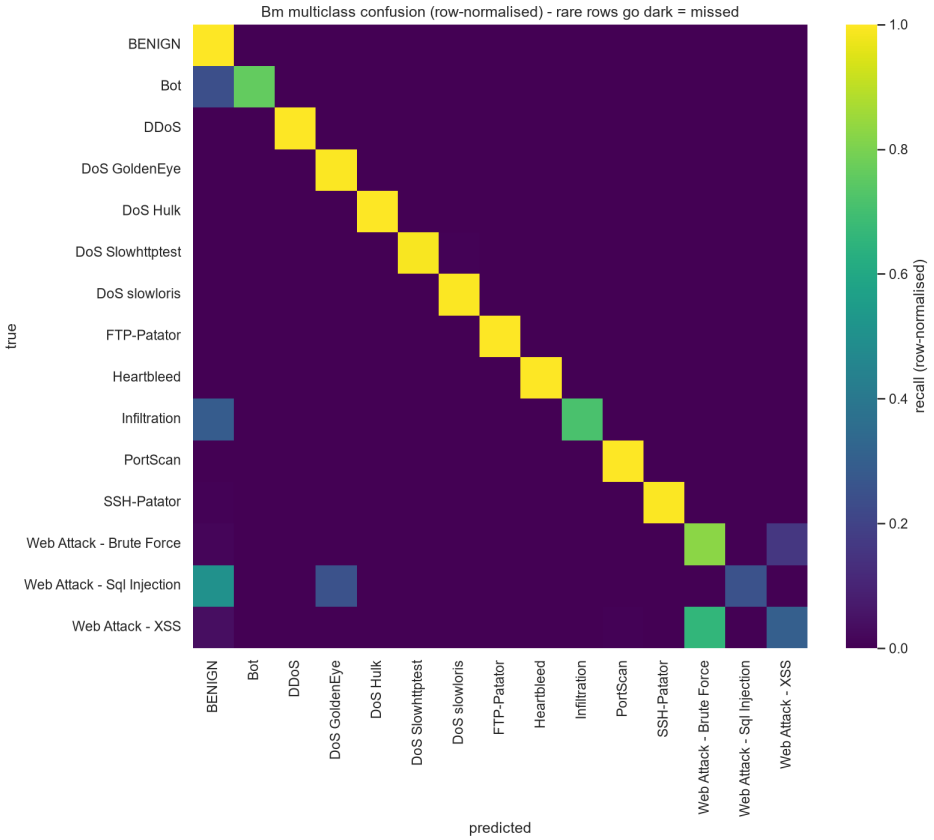

Figure 7: Row-normalised multiclass confusion matrix. Bright diagonal cells are recovered classes; dark rare-attack rows are systematically missed.

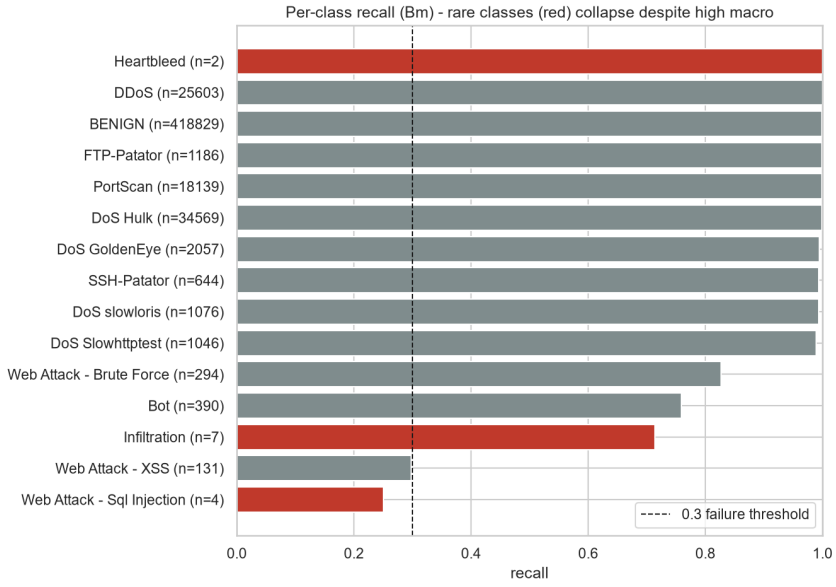

Figure 8: Per-class recall (rare classes in red). High macro/aggregate numbers coexist with near-zero recall on dangerous, rare attacks.

Cybersecurity implication and the FP/FN trade-off. Read as one day of traffic, the honest model raises a non-trivial alert queue while still missing the majority of the day's attacks. In SOC terms, an analyst trusting the source's 0.999 would expect near-total coverage; in reality

the detector misses most novel attacks. Because a missed intrusion (FN) is far costlier than a false alarm (FP), the right operating point sits high on the recall axis — which on this honest split means accepting poor precision, or (better) using anomaly detection and frequent retraining rather than a supervised classifier alone.

Friday (one day of traffic), honest RF-B0	**count**
Flows processed	703,245
Real attacks present	220,656
Alerts raised	21,636
of which false alarms	65 (0.3% of the alert queue)
Real attacks missed (false negatives)	199,085 (90.2% of attacks)

Table 4: SOC cost on a single day. The honest detector barely false-alarms (high precision) yet misses 90% of the day's attacks — the operational opposite of the "0.999" impression.

8 Conclusions

Key findings. (1) The 0.999 headline reproduces under the source's protocol but is an artifact of in-distribution memorisation. (2) Duplicate leakage, contrary to the common narrative, is *not* the inflation lever here (+0.03 pp); attack-type overlap in a random split is. (3) An honest temporal split drops macro-F1 by 51 pp because Friday's attacks are novel. (4) Aggregate metrics hide that rare classes are un-evaluable.

Lessons learned. Evaluation design dominates model choice on this dataset: the same Random Forest scores 0.999 or 0.488 depending only on how the split is made. Intellectual honesty (reporting the duplicate null result) made the critique stronger, not weaker.

Strengths and weaknesses of the source. *Strengths*: a broad, peer-reviewed model comparison on a standard dataset. *Weaknesses*: no deduplication, a leakage-prone random split on temporally-structured data, and aggregate-only metrics.

Future improvements. Temporal or attack-type-disjoint evaluation; deduplication with leakage reporting; per-class recall with explicit handling of un-evaluable classes; recall-aware metrics (MCC, F_β, PR-AUC); anomaly detection for novel attacks; and periodic retraining to handle concept drift.

9 Executive Summary

Problem. Flow-based network intrusion detection on CICIDS2017 (2.83M flows, $\approx 80\%$ benign, 14 attack types over Mon–Fri).

Source. Rodríguez et al. (2022, *Sensors* MDPI) report Random-Forest F1 > 0.999 using a random split over all days concatenated, no deduplication, and a macro-F1-only headline.

Methodology of our study. We rebuilt their pipeline in Python/scikit-learn and ran it three ways — A0 (their protocol), A1 (their protocol + deduplication), B0 (honest temporal split) — plus XGBoost and an unsupervised Isolation Forest, with a full recall-aware metric suite and per-class analysis.

Main findings. A0 reproduces F1 = 0.9987; deduplication changes it by only +0.03 pp; the honest temporal split collapses macro-F1 to 0.4883 (MCC 0.254), a 51-pp drop driven by

100% novel Friday attack classes; rare attacks have 2–7 evaluable samples and are effectively undetected.

Were the author's claims supported? The number is reproducible, but the implied claim of a generalising detector is **not supported**.

Most important insight. On day-segregated data, a random split measures memorisation, not detection; evaluation design, not the model, produces the 0.999.

Do we recommend reusing this approach on similar problems? No — not without temporal/novel-attack-aware evaluation, deduplication with leakage reporting, per-class recall, and recall-aware metrics. With those corrections the *workflow* is sound; the original *evaluation* is not.

Final conclusion. The headline is an evaluation artifact; the honest performance is modest, and the project demonstrates why methodological rigor matters more than a single impressive number — consistent with Engelen et al. (2021) [2] and Lanvin et al. (2022) [3].

10 Summing It Up

This project critically evaluated a peer-reviewed claim that Random Forest attains F1 $>$ 0.999 for flow-based intrusion detection on CICIDS2017. We reproduced the number under the original protocol, then demonstrated — with our own controlled experiments — that it does not survive sound evaluation: deduplication leaves it essentially unchanged (so the common "duplicate leakage" explanation is, here, a footnote), an honest temporal split removes 51 percentage points because the test day's attacks are novel, and aggregate metrics conceal that rare attacks cannot be evaluated. The objective was not to critique our own implementation but to assess the validity of the original work; our verdict is that the author's conclusions, while reproducible, are *not justified* as evidence of a deployable detector. A successful outcome here is the rigorous, honest analysis itself — whether or not the original claim is confirmed.

A Reproducibility & Environment

The full study runs from a pinned environment with one command; all randomness is fixed via `RANDOM_STATE = 42`.

Item	Value
Python	3.12
Environment manager	`uv` (pinned in `uv.lock`)
Core libraries	scikit-learn 1.9, XGBoost 3.3, imbalanced-learn 0.14, pandas, numpy
Notebook execution	`jupyter nbconvert -execute` — 0 errors, 0 warnings, 16 figures
Linting	`ruff check .` — clean
Unit tests	`pytest` — 16 passing (`tests/`, dataset-independent)
Report build	MiKTeX / `pdflatex` (this document)
Repository	`github.com/eyalsht/ids-cicids2017-critique` (public)

Table 5: Environment and reproducibility summary.

Reusable logic is factored into a tested package and mirrors the notebook's helpers:

- `src/ids_critique/data.py` — weekday parsing, whitespace/Inf cleaning, duplicate counting.

- `src/ids_critique/features.py` — skew detection, `log1p`, correlation pruning.
- `src/ids_critique/evaluate.py` — the IDS metric suite.
- `src/ids_critique/critique.py` — the flaw deltas (inflation, temporal drop, novel-class share).

B Rubric Traceability

Rubric component	Pts	Primary evidence
Problem Understanding & Source Selection	10	§1; README; `references/source_article.md`
Summary Quality	15	§1
Critical Evaluation of Claims	**20**	§2 (three flaws, deltas, verdict table)
Feature Engineering Analysis	10	§4; notebook §3; `features.py`
Exploratory Data Analysis	15	§3; notebook §2; Figures 3–4
Model Training & Comparison	15	§6; notebook §4; Figures 7–8
Evaluation & Error Analysis	10	§6–§7; notebook §5–§6
Code Quality & Software Engineering	5	`src/`, `tests/`, `uv`, ruff-clean, fixed seeds

Table 6: Mapping of each grading-rubric component to its primary evidence in this submission.

References

[1] M. Rodríguez, Á. Alesanco, L. Mehavilla, and J. García, "Evaluation of Machine Learning Techniques for Traffic Flow-Based Intrusion Detection," *Sensors*, vol. 22, no. 23, p. 9326, 2022. DOI: 10.3390/s22239326.

[2] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study," *2021 IEEE Security and Privacy Workshops (SPW)*, pp. 7–12, 2021. DOI: 10.1109/SPW53761.2021.00009.

[3] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and E. Totel, "Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes," *17th International Conference on Risks and Security of Internet and Systems (CRiSIS)*, 2022.

[4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," *4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.